

CO5 – AT1

Constraint-Based Problem Solving

SOURCE CODE

Q2. Exam Timetabling (Graph Coloring / NP-Complete Problem)

```
exams = ["Maths", "Physics", "DBMS", "OS", "Python", "Networks"]
```

```
# Student conflicts between exams
```

```
conflicts = [
```

```
    ("Maths", "Physics"),
```

```
    ("Maths", "DBMS"),
```

```
    ("Physics", "OS"),
```

```
    ("DBMS", "OS"),
```

```
    ("DBMS", "Python"),
```

```
    ("OS", "Networks"),
```

```
    ("Python", "Networks")
```

```
]
```

```
# Available time slots
```

```
slots = ["9:00-11:00", "11:30-1:30", "2:00-4:00"]
```

```
# Exam student strength
```

```
students = {
```

```
    "Maths": 40,
```

```
    "Physics": 35,
```

```
"DBMS": 50,

"OS": 45,

"Python": 30,

"Networks": 40

}

# Room capacities

rooms = {

    "Room A": 60,

    "Room B": 45,

    "Room C": 35

}

# Create graph

graph = {exam: [] for exam in exams}

for a, b in conflicts:

    graph[a].append(b)

    graph[b].append(a)

# Check whether a color can be assigned

def is_safe(exam, color, colors):

    for neighbor in graph[exam]:

        if colors.get(neighbor) == color:

            return False

    return True

# Backtracking graph coloring
```

```

def color_graph(index, colors):

    if index == len(exams):

        return True

    exam = exams[index]

    for color in range(len(slots)):

        if is_safe(exam, color, colors):

            colors[exam] = color

            if color_graph(index + 1, colors):

                return True

            del colors[exam]

    return False

# Generate timetable

colors = {}

if color_graph(0, colors):

    print("\n===== EXAM TIMETABLE =====")

    for slot_no, slot in enumerate(slots):

        print("\nTime Slot:", slot)

        for exam in exams:

            if colors[exam] == slot_no:

                assigned_room = None

                for room, capacity in rooms.items():

                    if students[exam] <= capacity:

                        assigned_room = room

```

```
        break

    if assigned_room:

        print(exam, "->", assigned_room,

              "(Students:", students[exam], ")")

    else:

        print(exam, "-> No suitable room")

else:

    print("No feasible timetable found.")

# Verify conflicts

print("\n===== CONFLICT CHECK =====")

valid = True

for a, b in conflicts:

    if colors[a] == colors[b]:

        print("Conflict:", a, "-", b)

        valid = False

if valid:

    print("No student conflicts found.")

    print("Timetable is valid.")
```